DIRECT PATH: continuous arm motion from the KR260 as fast as possible

This document: the shortest route to a continuously moving arm, with no intermediate deliverables. **Companion**: *kr260-measured-path-3s-and-continuous.pdf* (the full analysis: measurements, options, and the milestone plan that stops at ~4-5 s after one week before continuing to continuous). Every number here comes from that analysis (measured on the real checkpoint and synthesized on the K26 on 2026-10-22, reviewed); this page only re-orders the work for the single goal. For psamin and the FPGA side. 2026-10-22.

The requirement, in one line

LeRobot's client asks for the next chunk when 25 of the 50 actions (0.83 s) remain and discards actions that are already in the past when the chunk arrives, so the arm moves continuously only if a chunk's observation-to-queue latency is **<= 0.82 s with one inference in flight, <= 1.1 s with two, <= 1.2 s with three** (the chunk covers 1.67 s; the bound is the chunk horizon, not the FPGA). Today: 24.4 s. Target: **<= 1.1 s per chunk end to end, with two inferences in flight**.

Why the target is ~1.1 s and not 3 s

3 s per chunk gives 0.83 s of motion every 3 s (duty 28 %); 1.5 s gives 54 %; continuous needs the bound above. The only ways to relax it are a longer action horizon (chunk_size >= 181 actions: a different model, 3.6x the expert work) or executing actions slower than the 30 fps the policy was trained at (changes the dynamics). Neither is faster than meeting 1.1 s.

The budget at 1.1 s and how it is met (from the measured work and the synthesized costs)

Part of a chunk	Today (512 px, FP32 engine, ARM)	Direct design (256 px, int16 engine, kernels, sequencer)	What removes it
linear layers	9.0 s FPGA (255.7 GMAC at 128 FP32 MACs/clock)	~0.6 s (107 GMAC at 1,024 int16 MACs/clock, 200-225 MHz)	int16 engine (precision 0.00034 measured) + 256 px (2.4x fewer MACs)
vision attention	7.5 s ARM (38.7 GMAC)	~0.05 s on the engine (2.4 GMAC at 256 px)	256 px (16x less attention) + attention through the engine
softmax, GELU, norms, SiLU, residuals	~3.5 s ARM	~0.15 s FPGA kernels (fixed-point softmax 16 lanes, GELU table, norm/SiLU 4 lanes)	kernels
copies into/out of CMA, call overhead	3.5 s ARM (2.05 GB, 1,419 calls x ~2.5 ms)	~0.1 s (activations stay in FPGA-side buffers; ~400 commands per chunk from one list per layer)	command sequencer + fused calls
ARM glue (embeddings, flow update, Python)	~1 s	~0.3-0.5 s	sequencer; 3 torch threads; no numpy round trips
total	**24.4 s measured**	**~1.2-1.5 s estimated** (1.1 with 5 Euler steps or a 10 % faster clock)	

The estimate carries the cost model's uncertainty: the ARM glue (~1.2 s at 512 px today's model, less at 256 px) is the least certain term, the DDR bandwidth (14 GB/s) and the clock at 90 % DSP use are assumptions until the first build. If it lands at 1.3-1.5 s, three inferences in flight (bound 1.2 s) or num_steps 10 -> 5 (measured 0.114 vs the 10-step reference on the motion-onset frame; a task-success question) close the gap.

The work, in parallel from day 1 (owner in bold)

#	Track	Owner	Content	Days	Depends on
0	Decisions	**psamin**	(a) int16 arithmetic accepted (0.00034 measured vs 0.0043 today); (b) the 256-px re-finetune goes ahead; (c) the acceptance test = arm success rate on N episodes (the 3-frame gate cannot resolve below ~0.05 on the motion-onset frame)	0-1	-
1	256-px re-finetune	**psamin** (cluster)	retrain the expert with resize_imgs_with_padding = (256, 256), same recipe (~10k steps); frozen tower/LM unchanged (the position embedding is bucketized, so the 16x16 grid is valid); replay the frames, then the arm. Also fix the live pipeline's 256-px squash (client resizes to 256 before the policy upsamples to 512) so training and live inputs agree	2-4	0b
2	int16 engine	**FPGA side**	int16 W per-row scale x int16 X per-token scale, int48 accumulate (fabric accumulators for 2 token blocks per tile, or the DSP accumulator with one block), dequant in a 16-wide emit, overlapped X load, X tile packed 2 per URAM word, same bus interfaces (the IP is upgraded in place), C model + csynth; the 21 FP32-stored layers stay FP32 on the ARM. Started 2026-10-22 evening	3-5	0a
3	Kernels	**FPGA side**	attention through the engine with the heads of a layer batched per call; fixed-point row softmax (16 lanes); tanh-GELU table; LayerNorm/RMSNorm/SiLU; per-token X scale pass; patch-embedding conv as im2col GEMM - all as modes of the same float_top (no new IPs: this Vivado install cannot generate them)	10-15	2 (shares the emit/X paths)
4	Command sequencer	**FPGA side**	the ARM writes one command list per layer (engine + kernels back to back, activations staying in CMA); the driver issues ~400 lists per chunk instead of ~1,400 calls; two activation sets for two inferences in flight	4-6	2
5	Software	**FPGA side**	fused q/k/v and gate/up (exact), hoisted expert cross-attn K/V, language padding 'longest' (exact); client with two to three inferences in flight; selftest on the 256-px frames; deploy	3-4	1, 4
6	Builds and validation	both	Vivado builds (2 h each; plan 3-5 iterations for timing at 83-94 % DSP); board selftest; arm success-rate runs per 0c	ongoing	all

Critical path: tracks 2 -> 3 -> 4 -> 5 on the FPGA side (about 3-4 weeks of straight work with the builds), with track 1 finishing inside the first week so the 256-px checkpoint is ready when the engine is. **Earliest continuous arm**: ~4 weeks from the day psamin's decisions land, if the int16 engine closes timing within two builds and the re-finetune's task success holds. **What can be seen earlier**: the int16 engine on the board at the end of week 1 (~14-16 s at 512 px, ~4-5 s with the 256-px checkpoint) - not a milestone, just the first checkpoint of this plan.

What would shorten it further (each is psamin's call)

Lever	Effect on the 1.1 s budget	Cost
num_steps 10 -> 5	expert work and calls halved: ~-0.2 s	measured 0.114 vs the 10-step reference on the motion-onset frame; judge by task success
one camera	~-35 % of everything	re-finetune; likely worse task success (measured 1.9-2.0 without re-finetuning)
384 px instead of 256 px	+0.3 s (more MACs) but closer to the 512-px policy	re-finetune; 0.73 without
GPU/Jetson instead of the KR260	< 0.5 s per chunk out of the box	hardware; abandons the FPGA

Risks, in order

1. The 256-px policy may pick and place worse than the 512-px one (unknown until the arm run; the expert was trained on a 177-token prefix). 2. Timing closure of a 1,024-MAC int16 array at 83-94 % DSP use (every clock step lost costs ~10 % of the engine time; the budget tolerates one). 3. The X tile against the 64 URAMs (needs explicit 2-per-word packing). 4. The ARM glue estimate (0.3-0.5 s) is unmeasured; the sequencer is what controls it. 5. Sensor noise: in exact FP32 one 8-bit level of camera noise moves the motion-onset frame's chunk by 0.11-0.16, so the policy's own variability is larger than any arithmetic effect discussed here; the acceptance test must be the arm.